
SIGAI-SES: Sistema Integral de Gestión de Activos e Inventario — Securitas

Securitas Colombia S.A.



SIGAI-SES: Sistema Integral de Gestión de Activos e Inventario

— Securitas

Visión General del Sistema

[!IMPORTANT] **SIGAI-SES** es una plataforma empresarial robusta diseñada para la gestión, control y trazabilidad de activos tecnológicos, inventarios y procesos de garantías para el área de **Seguridad Electrónica de Securitas Colombia**.

El sistema centraliza la operación de bodegas, laboratorios y técnicos en campo, reemplazando los **procesos manuales basados en Excel** por flujos de trabajo automatizados.

Capacidades Clave

Capacidad	Descripción
Visión 360°	Ciclo de vida de cada equipo: desde compra, instalación, desmonte, hasta disposición final
Kardex Digital Universal	Historial inmutable de cada movimiento de inventario
Importación Inteligente	Procesamiento de Excel con <i>upsert</i> y normalización automática
Alertas Automáticas	Stock crítico, garantías estancadas, vencimientos
Actas Digitales	Generación de PDF con firma táctil desde dispositivos móviles
Auditoría Completa	Registro detallado de cada acción con valores anterior/nuevo

Stack Tecnológico Detallado

Backend (Servidor de Aplicaciones)

Tecnología	Versión	Propósito
	3.12+	Lenguaje base del backend
	0.136.1	Framework web asíncrono de alto rendimiento
	0.47.0	Servidor ASGI
	2.0.49	ORM asíncrono con aiomysql
	1.18.4	Migraciones de base de datos
	2.13.4	Validación de datos (schemas)
	2.14.1	Configuración con .env
	8.0+	Base de datos relacional
	3.5.0	Tokens JWT (HS256)
	4.0.1	Hashing de contraseñas con bcrypt
	3.0.3	Procesamiento de Excel
	3.1.5	Lectura de archivos Excel
	3.2.9	Exportación Excel con streaming (constant_memory)
	4.5.1	Generación de PDF con diseño corporativo
	12.2.0	Procesamiento de imágenes
	0.1.9	Rate limiting
	3.2.1	Logging estructurado JSON
	6.1.0	Monitoreo del sistema (/metrics)
	0.28.1	Cliente HTTP para tests
	9.0.3	Testing

Frontend (Interfaz de Usuario)

Componente	Tecnología	Versión
Framework		18.2
Lenguaje		5.2
Bundler		5.2
Estilos		3.4
Ruteo		6.22
Datos		5.101
HTTP		1.6.8
Gráficos		2.12
Iconos		0.363
Formularios		7.51
Firmas		1.1
Excel		0.18.5

Características Principales

Gestión de Inventario de Alto Nivel

- Control **multiregional** con datos aislados por ciudad
- Trazabilidad individual por **serial** y **placa de activo fijo**
- Asignación de **EPP** con fechas de vencimiento
- Importación masiva desde **Excel** con detección automática de tipo

Ciclo de Vida de Garantías

Estado	Progreso
Registrado	→
Enviado Proveedor	→

Estado	Progreso
Recibido Proveedor	→
Resuelto	→
Entregado Cliente	

- Numeración automática **GSES-XXX**
- Alertas de estancamiento a los **15 días** sin avance
- Vinculación con proveedores y **RMA**

Inteligencia y Automatización

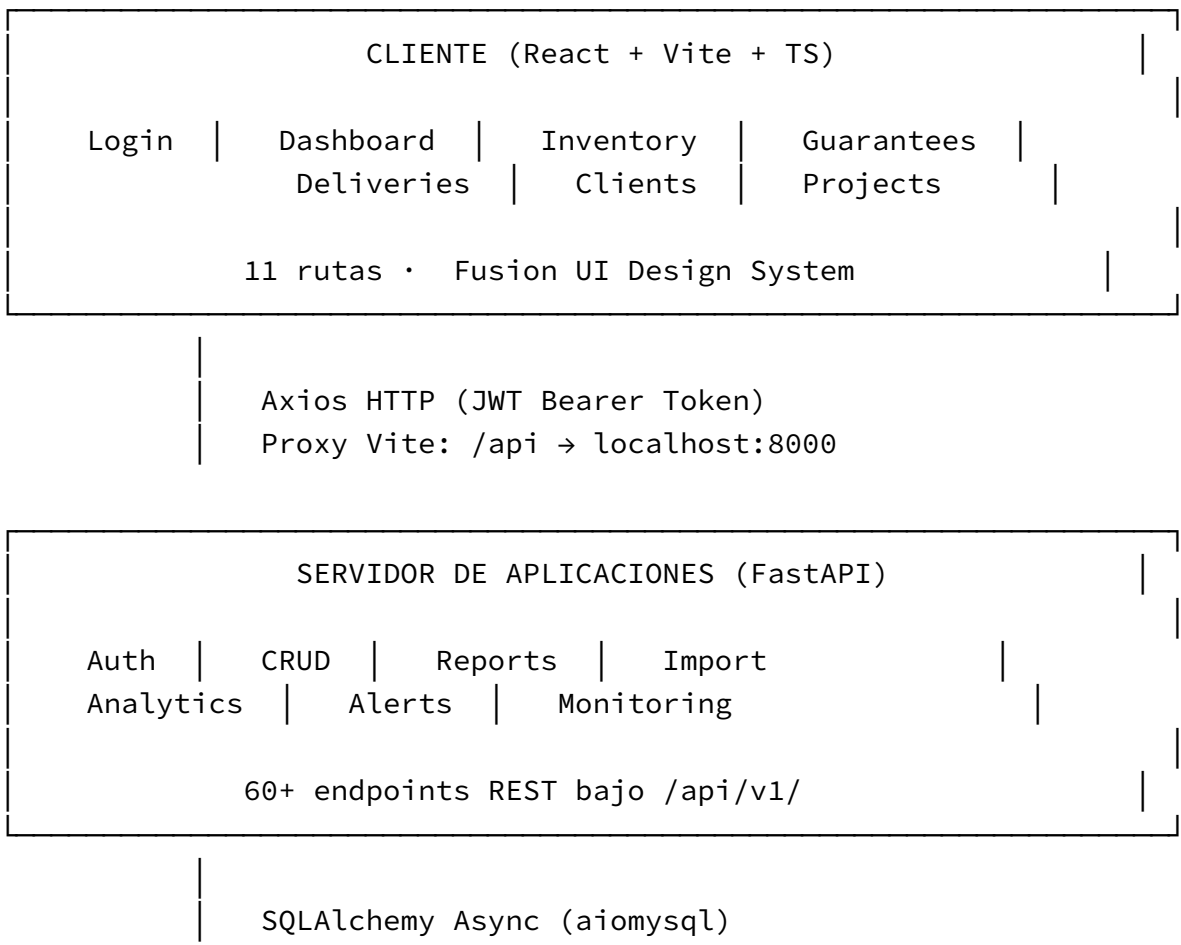
Característica	Descripción
Motor de Importación	Upsert con lógica transaccional
Motor de Alertas	APScheduler — evaluación cada 15 min
Reglas Predefinidas	stock_bajo · vencimiento · sin_movimiento · sobrestock
Streaming	Reportes de +30,000 registros sin degradación
Búsqueda Global	Respuesta a partir de 2 caracteres

Seguridad y Auditoría

Mecanismo	Estado
OAuth2 Password Flow	
JWT (access + refresh tokens)	

Mecanismo	Estado
RBAC — 3 roles	ADMIN • TECNICO • TECNICO_LABORATORIO
Hash bcrypt	
CORS restringido	
Auditoría con valores anterior/nuevo	
Rate limiting en login	10 req/min
GZip compression	> 500 bytes

Arquitectura del Sistema (3 Capas)



| Pool: 30 conexiones, timeout 60s

BASE DE DATOS (MySQL 8.0)				
usuarios	items	activos	garantias	
audit_logs	alerts	clientes	...	
18 tablas · Vistas: v_stock_consolidado, v_dashboard_kpis				

Estructura del Backend

Click para expandir la estructura completa del backend

Backend/

└─ app/	
└─ main.py	# Punto de entrada FastAPI
└─ core/	
└─ config.py	# Configuración (Settings con .env)
└─ security.py	# JWT + bcrypt helpers
└─ db/	
└─ session.py	# AsyncSession, engine, Base declarativa
└─ models/	# Modelos SQLAlchemy (18 tablas)
└─ user.py	# Usuario, Regional, SesionUsuario
└─ inventory.py	# Item, Activo, StockBulk, Movimiento...
└─ business.py	# Cliente, Proveedor, Proyecto
└─ deliveries.py	# ActaEntrega, DetalleActaEntrega
└─ guarantees.py	# Garantia
└─ alerts.py	# Alert, AlertRule
└─ audit.py	# AuditLog
└─ schemas/	# Schemas Pydantic (validación)
└─ auth.py	# Token, TokenData
└─ user.py	# UsuarioCreate/Update/Read
└─ inventory.py	# Item, Activo, Movimiento, EPP...

└─ business.py	# Cliente, Proyecto, Proveedor...
└─ deliveries.py	# ActaEntrega, DetalleActa
└─ alerts.py	# AlertRead, AlertUpdate
└─ audit.py	# AuditLog
└─ tracking.py	# EPP, HistorialUbicacion
└─ crud/	# Lógica de BD asíncrona
└─ crud_user.py	
└─ crud_inventory.py	
└─ crud_business.py	
└─ crud_alerts.py	
└─ crud_analytics.py	
└─ crud_audit.py	
└─ crud_deliveries.py	
└─ api/	
└─ deps.py	# Dependencias (get_current_user...)
└─ endpoints/	# Handlers de rutas (10 módulos)
└─ auth.py	# Login, refresh, logout...
└─ users.py	# CRUD usuarios, audit, settings...
└─ inventory.py	# CRUD items, activos, ubicaciones...
└─ business.py	# CRUD clientes, proyectos...
└─ analytics.py	# Summary, search global
└─ reports.py	# Exportación Excel/PDF
└─ alerts.py	# CRUD alertas, resumen...
└─ regionales.py	# Listado de regionales
└─ import_data.py	# Importación Excel (3 formatos)
└─ monitoring.py	# Health check, metrics
└─ services/	
└─ import_service.py	# Motor de importación Excel
└─ migrations/	# 9 versiones de migraciones Alembic
└─ tests/	
└─ test_import_service.py	# Tests del motor de importación
└─ scripts/	
└─ init_db.py	# Inicialización de BD + seed admin
└─ seed_admin.py	# Creación de usuario admin
└─ scheduler_alerts.py	# Evaluación programada de alertas
└─ trigger_alerts.py	# Evaluación manual de alertas
└─ backup_db.py	# Backup con mysqldump
└─ Base_de_datos/	

└─ sigai_ses_db.sql # Dump completo de BD (4941 líneas)

Estructura del Frontend

Click para expandir la estructura completa del frontend

```
Frontend/src/
├─ main.tsx # Punto de entrada
├─ App.tsx # Router principal con 11 rutas
├─ index.css # Estilos globales + Tailwind + CSS variables
├─ custom.d.ts # Declaraciones de tipos
├─ context/
│   └─ AuthContext.tsx # Contexto de autenticación JWT
├─ lib/
│   └─ toast.ts # Sistema de notificaciones
├─ hooks/
│   └─ useInventory.ts # Hook personalizado (React Query)
├─ services/ # Capa de API (axios con interceptors)
│   ├── api.ts # Instancia axios + refresh token
│   ├── auth.ts # Login, register, me
│   ├── users.ts # CRUD usuarios
│   ├── inventory.ts # Items, activos, ubicaciones, importación
│   ├── business.ts # Clientes, proyectos, garantías, actas
│   ├── analytics.ts # Dashboard stats, search global
│   ├── alerts.ts # Alertas
│   ├── desmontes.ts # Triage
│   └─ regionales.ts # Regionales
├─ components/ # Componentes reutilizables
│   ├── Fusion.tsx # Design System completo
│   ├── DashboardComponents # StatCard, QuickAccessBtn...
│   ├── ProtectedRoute.tsx # Ruta protegida (requiere token)
│   ├── RoleProtectedRoute.tsx # Ruta protegida por rol
│   ├── Toaster.tsx # Portal de notificaciones toast
│   ├── ExportMenu.tsx # Exportación PDF/Excel
│   └─ UserSettingsModal.tsx # Modal de preferencias
```

└─ pages/	# 11 páginas
├─ Login.tsx	# Inicio de sesión
├─ Dashboard.tsx	# Panel principal con KPIs
├─ Inventory.tsx	# Gestión de inventario
├─ Clients.tsx	# Directorio de clientes
├─ Projects.tsx	# Gestión de proyectos
├─ Guarantees.tsx	# Seguimiento de garantías
├─ Alerts.tsx	# Centro de alertas
├─ Audit.tsx	# Bitácora de auditoría (solo ADMIN)
├─ Users.tsx	# Administración de usuarios (solo ADMIN)
├─ Deliveries.tsx	# Actas de entrega (solo ADMIN)
└─ Desmontes.tsx	# Triage de equipos desmontados

Instalación Rápida

[!WARNING] Asegúrate de tener los siguientes requisitos antes de comenzar:

Requisito	Versión Mínima
Python	3.12+
Node.js	18+
MySQL/MariaDB	8.0+ / 10.5+

Backend

```
# 1. Clonar e ir al directorio
cd Backend
```

```
# 2. Crear entorno virtual
python -m venv .venv
```

```
# 3. Activar entorno
# Windows:
.venv\Scripts\activate
# Linux/Mac:
```

```
source .venv/bin/activate

# 4. Instalar dependencias
pip install -r requirements.txt

# 5. Configurar .env (ver .env.example)

# 6. Iniciar servidor
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

Frontend

```
# 1. Ir al directorio
cd Frontend

# 2. Instalar dependencias
npm install

# 3. Configurar .env
# VITE_API_BASE_URL=http://localhost:8000/api/v1

# 4. Iniciar en desarrollo
npm run dev

# 5. Generar versión de producción
npm run build
```

Estado del Proyecto

Click para ver indicadores del proyecto

Indicador	Estado
Versión Actual	v1.0.0 — Release Candidate
Fecha	Julio 2026
Cobertura de Tests	Motor de importación
Endpoints Documentados	60+ REST

Indicador	Estado
Tablas en BD	18
Roles de Usuario	3 (ADMIN, TECNICO, TECNICO_LABORATORIO)

SIGAI-SES — Sistema Integral de Gestión de Activos e Inventario Securitas Colombia S.A. —
Unidad de Seguridad Electrónica (SES) v1.0.0 · Julio 2026